Teaching LLMs Expert Judgment: A Chess RL Case Study

Ibrahim El-Fayoumi · Perth Machine Learning Group · Microsoft, 100 St Georges Terrace, Perth

The hook: prompting can't transfer judgment

- Frontier LLMs, six routine investor tasks, prompt that just states the task: ~50% accuracy
- Experts rewrite the prompts, automatic prompt optimisation on top: mid-70s - still below the 80% trust threshold
- "An explicit prompt can only convey the intuition an expert is able to put into words"
- The judgments that matter most are the hardest to articulate
  - *Source: Thinking Machines Lab x Bridgewater AIA Labs, 'Learning to Replicate Expert Judgment in Financial Tasks', 30 Jun 2026*

THE THESIS

Thesis: tacit judgment is trained in, not prompted

- Expert judgment is tacit - it lives in thousands of small decisions, not in a written rubric
- Fine-tuning on expert signal lets the model develop its own judgment
- A small model trained on expert signal can beat a much larger frontier model
- We need a testbed where 'expert judgment' is objective, cheap, and measurable
- **That testbed is chess: Qwen3-1.7B trained against Stockfish**

SETUP: CHESS AS THE CONTROLLED TESTBED

Setup: Qwen3-1.7B + Stockfish reward

- Policy: Qwen3-1.7B - small enough to train on modest compute, large enough to produce legal play
- Teacher/reward: Stockfish 17 - an expert ~1000+ Elo above anything the policy can reach
- Pipeline: SFT on game transcripts -> GRPO with engine reward -> off-policy extension (LUFFY)
- Evaluated on held-out positions (n=1000) + full games vs Stockfish and vs random

How we measure

- Legal-move rate: can the model play the game at all? (formats, state tracking)
- Median centipawn loss (CPL): quality of each move vs Stockfish's best
- **Use the MEDIAN, not the mean - the mean is dominated by rare blunders (std ~2900 cp)**
- Game outcomes vs Stockfish (20 games) and vs random (10 games); avg game length
- Estimated Elo

STAGE 1 - SFT ON GAME TRANSCRIPTS

Stage 1 - SFT: what the model learns

- Data: game transcripts (FEN state -> move in SAN), generated with Stockfish
- Learns: move format, board syntax, opening patterns, 'what a reasonable move looks like'
- Does NOT learn: why a move is good - no signal beyond imitation
- Result: 100% legal-move rate on the test set - the model can play
- But: median CPL 81 cp, loses 0-20 vs Stockfish (20/20), 0 wins vs random (9 draws, 1 loss)

STAGE 2 - GRPO WITH STOCKFISH AS THE REWARD

Stage 2 - GRPO: reward from the engine

- GRPO: critic-free policy gradient - sample G moves per state, reward each, normalise within the group
- Reward: Stockfish evaluation of the chosen move (centipawn signal), plus game outcome
- Advantage is relative: 'better than the group's average move', not 'objectively good'
- No value network to train - fits our compute budget
- Reward shaping: legality penalty, engine eval, game result

Failure modes: sparse reward, brutal credit assignment

- One terminal reward spread over ~30 decisions per game - credit assignment is the core difficulty
- Long-horizon strategy: a good move's value may only appear 20 plies later
- Strategy has no token signature - nothing in the trace says 'this was the strategic move'
- Reward hacking pressure: engine-eval shaping can be gamed locally (shallow tactics)
- Off-policy drift: by the time a game finishes, the policy that played it is stale

Strategic vs tactical: why chess is hard for a 1.7B model

- Tactics (forks, pins): local, 2-4 plies, pattern-shaped - SFT handles these
- Strategy (piece activity, pawn structure, plans): global, 20+ plies, no local pattern
- Branching: ~35 legal moves/ply - the horizon problem explodes combinatorially
- A frontier model 'knows' chess commentary; a 1.7B must learn search-free evaluation
- Humans also can't articulate strategic judgment - exactly the tacit-knowledge problem

GRPO results - the honest read

- *Sources: evaluation_results/comparison.json (canonical, frozen); benchmark_results.json is a stale second rl/final run - not used*

Metric (1000 held-out positions)	SFT	GRPO (RL)
Legal-move rate	100%	100%
Median centipawn loss	81.0	90.0
Mean centipawn loss (blunder-dominated)	1045.7	1022.7
vs Stockfish (20 games)	0-20	0-20
Avg game length (plies)	24.3	27.7
vs random (10 games)	0W / 9D / 1L	0W / 9D / 1L

So did RL teach anything new?

- Headline evals flat - but behaviour changed: longer games, different move distributions
- RL sharpened and stabilised SFT behaviour rather than adding new capability (so far)
- Consistent with the literature: RL mostly reweights what SFT already knows
- The gap is signal density, not model capacity - one reward over ~30 decisions
- **Which sets up stage 3: denser signal from the same compute**

STAGE 3 - OFF-POLICY LEARNING: LUFFY AND DISTILLATION

LUFFY: off-policy guidance from a stronger teacher

- LUFFY: mix a strong teacher's traces into GRPO rollouts, with policy-aware down-weighting
- Learns from the expert even where the policy's own samples are all bad
- Regularisation keeps the mixture on-policy (importance-based advantage adjustment)
- Extra learning from limited compute - we don't need more engines, we need better use of the one we have

Why on-policy distillation helps

- Exposure bias: off-policy training never fixes the states the policy actually reaches
- Dense per-token signal vs one terminal reward - credit assignment becomes trivial
- The student's own rollouts stay on-policy; only the TARGET comes from the teacher
- Cheaper than sampling from a frontier teacher at every state

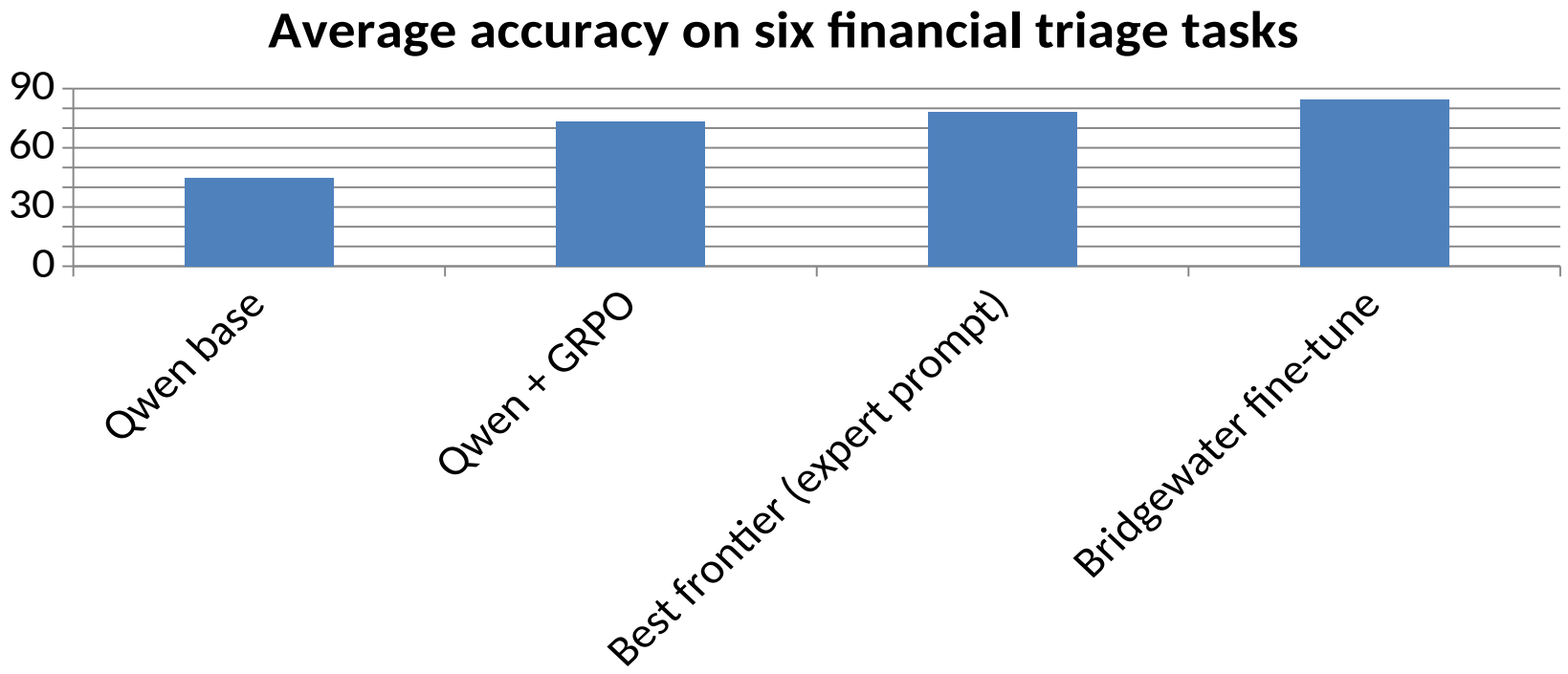
EVIDENCE AT SCALE: THINKING MACHINES X BRIDGEWATER

The same recipe, at production scale

- Bridgewater AIA Labs + Thinking Machines Lab, published 30 Jun 2026
- Task: six document-triage tasks from real investor workflows (relevancy, labelling, truncation)
- Base: Qwen3-235B fine-tuned on Tinker - GRPO start, then interleaved batching + CISPO + on-policy distillation
- Expert labels only where needed: contested examples routed to investors for re-review
- Result: fine-tuned open model beats every frontier model tested - at 13.8x lower inference cost per task

Bridgewater numbers

- *84.7% vs 78.2% = 29.8% fewer errors; 13.8x lower inference cost per task. Source: TML/Bridgewater post, 30 Jun 2026 (thinkingmachines.ai)*



Caveats (say them before the audience does)

- Company-run internal evaluation - not an independent public benchmark
- Six tasks, private test set cleared for release; generalisation beyond them unproven
- Cost figure covers inference only - expert labelling and training cost not included
- Frontier models not improving fast on this axis: GPT 5.4 cost 43% more than 5.2 for marginal accuracy gain
- Same discipline applies to OUR numbers: small n, one seed, self-reported

Key questions this testbed answers

- **How small can a model be and still reason?**
  - 1.7B reaches 100% legal play and club-level tactics after SFT; strategic play needs better signal, not more parameters
- **Does RL teach new capabilities or sharpen SFT?**
  - In our run: sharpening (longer games, stable behaviour), evals flat - consistent with TML/Bridgewater where the recipe, not scale, moved the number
- **Reward from an engine ~1000 Elo stronger - what goes wrong?**
  - Reward signal too sparse in game outcome; per-move engine shaping helps but invites local (tactical) reward hacking; the stronger the teacher, the more its best moves are unreachable-by-imitation

Takeaways

- Expert judgment is tacit - prompting plateaus (~50-78%), fine-tuning on expert signal crosses the trust threshold (84.7%)
- Chess is a clean rehearsal space: real reward design, real credit-assignment pain, honest metrics
- Small models + expert signal beat large models + prompts, at a fraction of the cost
- On-policy distillation turns a sparse terminal reward into dense per-token learning
- The bottleneck is signal quality, not model size

Demo & materials

- Notebook: qwen-chess-rl - GRPO loss & on-policy distillation, step by step (DAT-341)
- Demo: full game, trained policy vs Stockfish 17, per-ply eval + PGN (offline-capable)
- Repo: github.com/deep-science-ai/qwen-chess-rl
  - *[QR code placeholder - final slide, add before dry run]*
  - Demo fallback: recorded game + figure committed in the notebook (no live GPU needed)

Thank you - questions?

- Repo: github.com/deep-science-ai/qwen-chess-rl
- References:
  - Thinking Machines Lab x Bridgewater AIA Labs - 'Learning to Replicate Expert Judgment in Financial Tasks' (30 Jun thinkingmachines.ai/news/learning-to-replicate-expert-judgment-in-financial-tasks
  - LUFFY: off-policy RL with mixing guidance (arXiv 2504.14945)
  - On-policy distillation - Thinking Machines Lab blog
- Ibrahim El-Fayoumi - Perth Machine Learning Group

Scan for slides, code & the step-by-step notebook: `github.com/deep-science-ai/qwen-chess-rl`